

Rendszerterv, Felhasználói Kézikönyv és Döntési Napló

1. Rendszerterv

1.1. A rendszer célja

A ParkingManager egy konzol alapú parkolóhely-kezelő és foglalási szoftver. Biztosítja az ütközésmentes foglalást és a meglévő foglalások kezelését.

1.2. Architektúra és tervezési minták

- **UI:** Konzolos menürendszer C# strukturált metódusok felhasználásával (Program.cs).
- **Üzleti logika :** ParkingService osztály, amely felelős a foglalási szabályok, a dátumok validációjának és a különböző parkolóhely-típusokhoz kapcsolódó üzleti korlátok (pl. mozgáskorlátozott igazolvány, elektromos autók 3 órás időorlátja) ellenőrzéséért.
- **Adatelérés:** ParkingRepository osztály, amely a Dapper segítségével kommunikál az adatbázissal. Absztrakcióját az IParkingRepository-val biztosítottam.
- **Gyár programtervezési minta:** Az IDbConnectionFactory és OracleConnectionFactory osztályok felelnek az adatbázis-kapcsolatok példányosításáért, teljesen leválasztva a technológiai sajátosságokat az adatelérésről.

1.3. Adatbázis-terv (Oracle)

A rendszer egy Oracle adatbázissal kommunikál, amelynek főbb táblái:

- **ParkingSpots:** Tárolja a parkolóhelyeket. Mezők: Id (PK, Identity), Code (egyedi kód, pl. A-01), SpotType (STANDARD, DISABLED, EV_CHARGING, VIP).
- **Reservations:** Tárolja a foglalásokat. Mezők: Id (GUID string), ParkingSpotId (FK), RequesterEmail, StartTime, EndTime, IsActive (logikai törléshez).

1.4. Konkurencia- és Tranzakciókezelés

Új foglalás rögzítésekor a tranzakció egy explicit Oracle FOR UPDATE zárat használ a cél parkolóhelyre, így megelőzve, hogy két felhasználó egyszerre ugyanarra a helyre foglaljon.

2. Felhasználói Kézikönyv

2.1. Rendszerkövetelmények

- Docker és Docker Compose (a futtatáshoz).

2.2. A Rendszer Indítása (Egyparanccsal)

A következő paranccsal indítható a projekt könyvtárából:

```
docker-compose run --rm parking-app
```

Ez a parancs felépíti az adatbázist, majd elindítja az interaktív konzolos menürendszert.

2.3. Menüpontok és Használat

A program elindulása után az alábbi menüpontok érhetők el:

1. **Parkolóhelyek listázása:** Kilistázza az összes parkolóhelyet a hozzájuk tartozó egyedi kóddal és típussal (STANDARD, DISABLED, EV_CHARGING, VIP).
2. **Adott hely foglalásainak lekérdezése:** Kilistázza az összes parkolóhelyet, majd onnan egy parkolóhely kiválasztásával, a kiválasztott parkolóhely foglalásait is kilistázza.
3. **Új foglалás rögzítése:**
 - Kilistázza az összes parkolóhelyet
 - Bekéri a parkolóhely azonosítóját, az igénylő e-mail címét, valamint a kezdő és záró időpontot (yyyy-MM-dd HH:mm).
 - Amennyiben a hely típusa DISABLED vagy EV_CHARGING, felteszi a jogosultságra vonatkozó kiegészítő kérdéseket (igazolvány megléte, elektromos jármű).
4. **Foglалás lemondása:** Az e-mail cím megadása után lekérdezi a felhasználó aktív foglалásait, majd sorszám alapján lehetővé teszi azok logikai törlését.
5. **Kilépés:** Bezárja az alkalmazást.

4. API Leírás

4.1. IParkingRepository (Adatelérési réteg szerződése)

Ez az interfész definiálja az adatbázis felé irányuló összes műveletet.

- Task<IEnumerable<ParkingSpot>> GetParkingSpotsAsync()
 - *Leírás:* Lekérdezi az összes parkolóhelyet a rendszerből a típusaik szerint.
- Task<IEnumerable<Reservation>> GetReservationsBySpotIdAsync(int spotId)
 - *Leírás:* Visszaadja egy adott parkolóhelyhez tartozó összes érvényes foglалást.
- Task<IEnumerable<Reservation>> GetActiveReservationsByEmailAsync(String email)
 - *Leírás:* Lekérdezi a megadott e-mail címhez tartozó aktív foglалásokat (pl. lemondáshoz).

- Task<ParkingSpot?> GetSpotByIdAsync(int id)
 - *Leírás:* Egyedi azonosító alapján visszaad egy parkolóhelyet, vagy null-t, ha nem létezik.
- Task<bool> CreateReservationAsync(Reservation reservation)
 - *Leírás:* Tranzakción belül, pesszimista zárolással megpróbálja rögzíteni az új foglalást. Visszatérési értéke true, ha sikeres, false, ha ütközés miatt meghiúsult.
- Task<bool> CancelReservationAsync(string reservationId)
 - *Leírás:* Logikai törlést hajt végre (IsActive = false) a megadott azonosítójú foglaláson.

4.2. ParkingService (Üzleti logika)

A ParkingService felelős a validációs szabályok betartatásáért, mielőtt a kérés eljutna az adatbázis-rétegbe.

- Task<(bool Success, String Message, Reservation? Reservation)> TryCreateReservationAsync(...)
 - *Validációs szabályok, amelyeket ellenőriz:*
 1. Az e-mail cím nem lehet üres.
 2. A záró időpontnak a kezdő időpont után kell lennie.
 3. Nem lehet múltbeli időpontot foglalni.
 4. A parkolóhelynek léteznie kell.
 5. Mozgáskorlátozott (DISABLED) hely esetén kötelező az érvényes igazolvány megléte.
 6. Elektromos töltő (EV_CHARGING) hely esetén kötelező a zöld rendszám, és a foglalás hossza nem haladhatja meg a 3 órát.
 7. Idősáv ütközés vizsgálata a Repository-n keresztül.
- Task<bool> CancelReservationAsync(string reservationId)
 - *Leírás:* Továbbítja a lemondási kérést a Repository felé.

4. Döntési Napló és Reflexió

4.1 Döntési Napló

#	Döntési Pont	Választott Megoldás	Miért ezt választottam? (Előnyök)	Elvetett Alternatíva
1	Versenyhelyzet/Konkurencia kezelése	Pesszimista zárolás tranzakcióban	Biztos, hogy nem keletkezik dupla foglalás.	Optimista zárolás/Ellenőrzés kihagyása. Adatvesztéshez vagy duplikált adatokhoz vezethetett volna
2	Architektúra rétegezése	Külön repository és service réteg	Az üzleti logika és az adatelérés szétválasztásával növeltem a tesztelhetőséget és a karbantarthatóságot	Az üzleti logika és az adatelérés egy osztályba helyezése.
3	Törlés módja	Logikai törlés	Megmaradnak az adatok időpont lemondása után is. Szükség szerint visszaállíthatóak. Könnyen készíthető statisztika.	Fizikai törlés. Az összes lemondott adat elveszik, nehezebbé teheti az esetleges hibaelhárításokat, elemzéseket

4.2 Reflexió

Számomra a feladat legmegterhelőbb része adatbázis-használat volt C#-ban, hiszen tanulmányaim során csak felületesen érintettük. Utánanéztem a Dapper használatának, ezzel elsajátítva annak alapjait. A következő kihívás a Oracle és a Dapper zökkenőmentes összekötése jelentette, amit a konténeres környezet helyes beállításával oldottam meg. AI-asszisztentstől a program szerkezeti felépítéséhez ,az osztályokban lévő hibák kijavításához, a versenyhelyzetek kezeléséhez, valamint a CLI megírásával kapcsolatban használtam.[\[AI-Chat link\]](#)